1. Create a Java class called PaymentProcessor with two overloaded methods processPayment(): one that takes only an amount, and one that takes amount and a coupon code. Print which version is called and the final amount in each case.

Method overloading means creating multiple methods with the same name but different parameters.

```
class PaymentProcessor {
  void processPayment(double amount) {
    System.out.println("Payment without coupon");
    System.out.println("Final Amount: " + amount);
  }

  void processPayment(double amount, String coupon) {
    double discount = 0;

    if (coupon.equals("SAVE10")) {
      discount = amount * 0.10;
    }

    System.out.println("Payment with coupon: " + coupon);
    System.out.println("Final Amount: " + (amount - discount));
  }

  public static void main(String[] args) {
    PaymentProcessor p = new PaymentProcessor();

    p.processPayment(1000);
    p.processPayment(1000, "SAVE10");
  }
```

```
}
```

Output:

Payment without coupon

Final Amount: 1000.0

Payment with coupon: SAVE10

Final Amount: 900.0

2. Build two classes, InstagramUploader and YouTubeUploader, each with a method uploadContent(). Both should extend a base class SocialMediaUploader and override uploadContent() to print a message showing how uploading works differently for Instagram and YouTube.

Method overriding means a child class provides its own implementation of a parent class method.

```
class SocialMediaUploader {
  void uploadContent() {
    System.out.println("Uploading content to social media");
  }
}

class InstagramUploader extends SocialMediaUploader {
  @Override
  void uploadContent() {
    System.out.println("Instagram: Uploading photos and reels");
  }
}

class YouTubeUploader extends SocialMediaUploader {
  @Override
```

```
  void uploadContent() {
    System.out.println("YouTube: Uploading videos");
  }
}

public class Main {
  public static void main(String[] args) {
    InstagramUploader instagram = new InstagramUploader();
    YouTubeUploader youtube = new YouTubeUploader();

    instagram.uploadContent();
    youtube.uploadContent();
  }
}
```

Output:

Instagram: Uploading photos and reels

YouTube: Uploading videos

3. Write a function in Java or Python that simulates a Flipkart-style search: overload a method searchProduct() to allow searching by product name or by product name and category. Demonstrate both usages with sample data.

This Java program searches for products by name or by both name and category.

```
class ProductSearch {
  void searchProduct(String name) {
    System.out.println("Searching product: " + name);
  }

  void searchProduct(String name, String category) {
```

```
        System.out.println("Searching product: " + name);
        System.out.println("Category: " + category);
    }

    public static void main(String[] args) {
        ProductSearch search = new ProductSearch();

        search.searchProduct("Laptop");
        System.out.println();

        search.searchProduct("Laptop", "Electronics");
    }
}
```

Output:

Searching product: Laptop

Searching product: Laptop

Category: Electronics

**4. Given this code: class MusicPlayer { void play(String song) { System.out.println("Playing: " + song); } } class SpotifyPlayer extends MusicPlayer { void play(String song) { System.out.println("Streaming on Spotify: " + song); } } — Create an object of type MusicPlayer but assign it a SpotifyPlayer instance, then call play(). Explain the output.

Hint: This tests runtime polymorphism (overriding) and dynamic method dispatch.**

```
class MusicPlayer {
    void play(String song) {
        System.out.println("Playing: " + song);
```

```
    }
}

class SpotifyPlayer extends MusicPlayer {
    @Override
    void play(String song) {
        System.out.println("Streaming on Spotify: " + song);
    }
}

public class Main {
    public static void main(String[] args) {
        MusicPlayer player = new SpotifyPlayer();

        player.play("Perfect");
    }
}
```

Output:

Streaming on Spotify: Perfect